

Introduction to Data Science

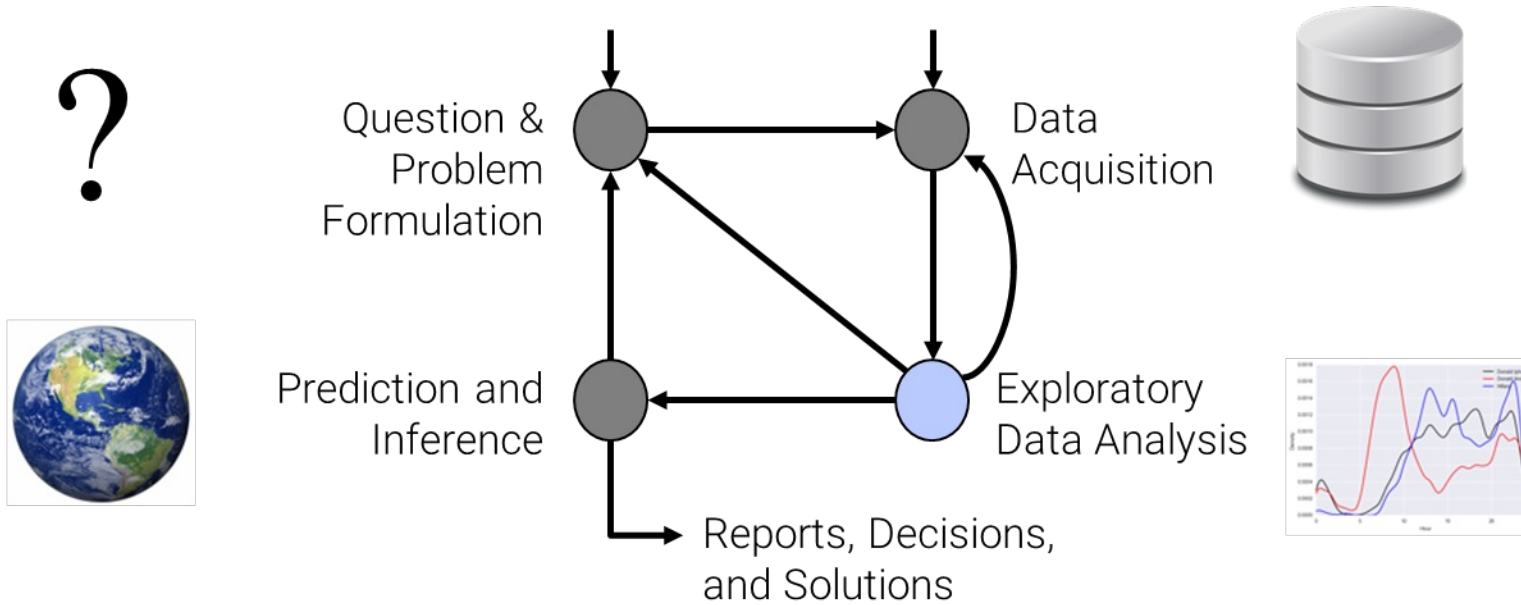
NumPy



Usman Nazir

Agenda

- What is NumPy?



NumPy

- The fundamental package for scientific computing with Python

NumPy

- The fundamental package for scientific computing with Python
- Powerful in N-dimensional arrays
 - Fast and versatile
 - The NumPy vectorization, indexing and broadcasting concepts are the de-facto standards of array computing.

NumPy

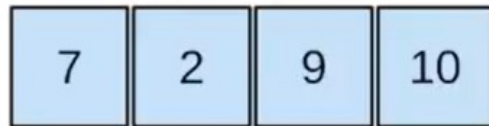
- The fundamental package for scientific computing with Python
- Powerful in N-dimensional arrays
 - Fast and versatile
 - The NumPy vectorization, indexing and broadcasting concepts are the de-facto standards of array computing.
- Numerical computing tools
 - Comprehensive mathematical functions
 - Random number generators
 - Linear algebra routines
 - Fourier transform and more.

NumPy

- The fundamental package for scientific computing with Python
- Powerful in N-dimensional arrays
 - Fast and versatile
 - The NumPy vectorization, indexing and broadcasting concepts are the de-facto standards of array computing.
- Numerical computing tools
 - Comprehensive mathematical functions
 - Random number generators
 - Linear algebra routines
 - Fourier transform and more.
- Open source

N-dimensional arrays

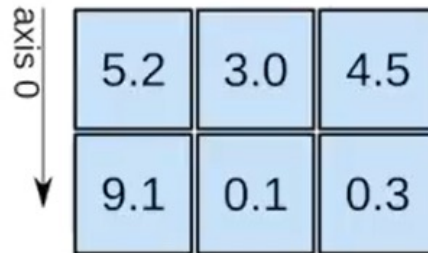
1D array



axis 0 →

shape: (4,)

2D array

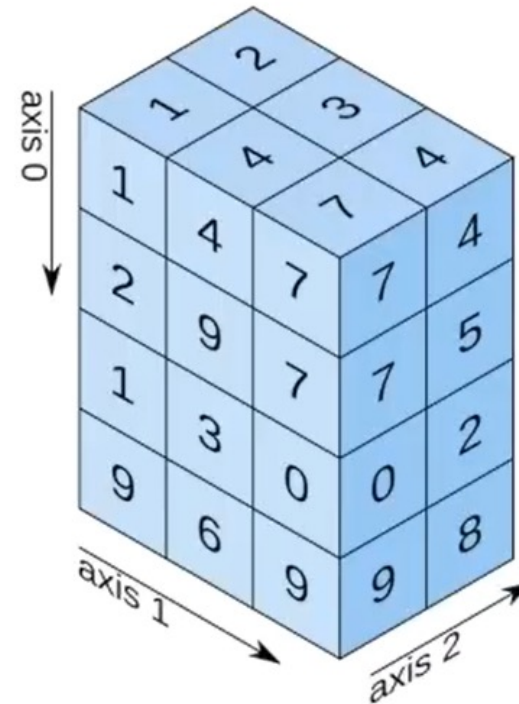


axis 0 ↓

axis 1 →

shape: (2, 3)

3D array



shape: (4, 3, 2)

Difference between Lists and NumPy arrays

Lists

- Slow
 - Size, reference count, object type, object value

NumPy

- Very fast
 - Fixed type
 - No type checking when iterating through objects
 - Int32, Int16, Int8

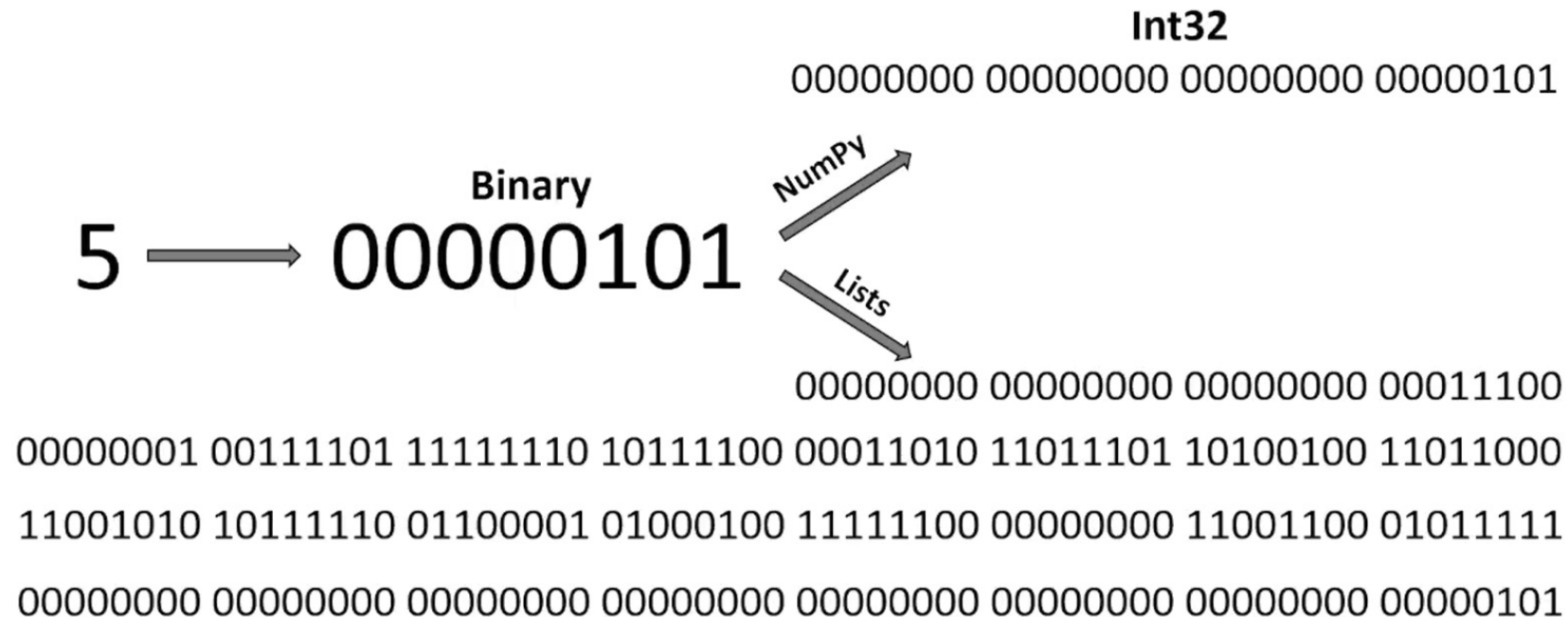
Difference between Lists and NumPy arrays

Lists

- Slow
 - Size, reference count, object type, object value

NumPy

- Very fast
 - Fixed type
 - Int32, Int16, Int8



Difference between Lists and NumPy arrays

Lists

- Slow
 - Size, reference count, object type, object value
- Not Contiguous memory

NumPy

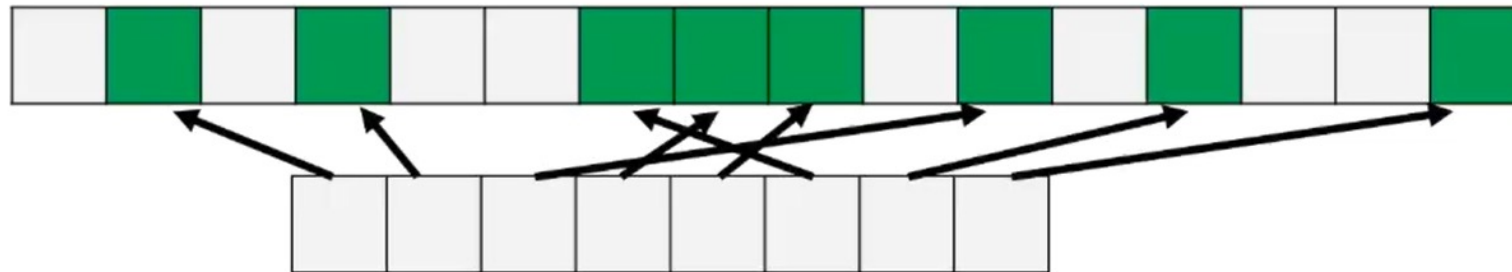
- Very fast
 - Fixed type
 - No type checking when iterating through objects
 - Int32, Int16, Int8
- Contiguous memory

Difference between Lists and NumPy arrays

Lists

- Slow
 - Size, reference count, object type, object value
- Not Contiguous memory

Lists



NumPy

- Very fast
 - Fixed type
 - No type checking when iterating through objects
 - Int32, Int16, Int8
- Contiguous memory

NumPy



Difference between Lists and NumPy arrays

Lists

- Slow
 - Size, reference count, object type, object value
- Not Contiguous memory

NumPy

- Very fast
 - Fixed type
 - No type checking when iterating through objects
 - Int32, Int16, Int8
- Contiguous memory
 - SIMD (Single Instruction Multiple Data)
 - Effective Cache Utilization

How are Lists different from NumPy?

Lists

- Insertion, deletion, appending, concatenation, etc.

NumPy

- Insertion, deletion, appending, concatenation, etc.
- Lots More 😊

How are Lists different from NumPy?

Lists

- Insertion, deletion, appending, concatenation, etc.
 - $A = [1, 3, 5]$
 - $B = [1, 2, 3]$
 - $A * B = \text{Error}$

NumPy

- Insertion, deletion, appending, concatenation, etc.
- Lots More 😊

How are Lists different from NumPy?

Lists

- Insertion, deletion, appending, concatenation, etc.
 - `A = [1, 3, 5]`
 - `B = [1, 2, 3]`
 - `A*B = Error`

NumPy

- Insertion, deletion, appending, concatenation, etc.
- Lots More 😊
 - `A = np.array([1, 3, 5])`
 - `B = np.array([1, 2, 3])`
 - `A*B = np.array([1, 6, 15])`

Applications of NumPy

- Mathematics (MATLAB replacement)
- Plotting (**Matplotlib**)
- Backend (**Pandas**, Connect 4, Digital Photography)
- Machine Learning applications' key library (Tensors)

Summary

- What is NumPy?